“Linux Service Supervision”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

PRAVALIKA	**2520030402**
KOMMALAPATI RISHITHA	**2520030202**
G.S. HASINI	**2500030238**

Operating Systems and Systems Programming (25CS2104E) | 2026–27, Term-I

Under the Esteemed Guidance of

Harika
Project Guide
Department of Computer Science and Engineering

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled “Linux Service Supervision” is a record of bona fide work carried out by Pravalika – 2520030402, Kommalapati Rishitha – 2520030202, and G.S. Hasini – 2500030238, as part of the Operating Systems and Systems Programming course during the academic year 2026–27, Term-I. The work presented in this report focuses on Linux service supervision using Bash shell scripting and systemd utilities, particularly systemctl, to inspect and report the state of selected system services. The report is prepared for academic submission and the technical sources consulted are acknowledged.

Pravalika	**2520030402**
Kommalapati Rishitha	**2520030202**
G.S. Hasini	**2500030238**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the project report entitled “Linux Service Supervision” is a bona fide work carried out and submitted by Pravalika (2520030402), Kommalapati Rishitha (2520030202), and G.S. Hasini (2500030238), in partial fulfillment of the requirements of the Operating Systems and Systems Programming course at K L (Deemed to be) University during the academic year 2026–27, Term-I. The report describes a Linux-based service supervision utility using Bash and systemd service-management commands.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

We express our sincere gratitude to everyone who supported and guided us during the completion of our project, “Linux Service Supervision.” This project helped us understand how Linux manages background services and how shell scripting can automate routine service-status checks.

We are especially grateful to our project guide, Harika, for the guidance, encouragement, and academic support provided throughout the project. Her direction helped us understand Linux service management, systemd, systemctl commands, Bash scripting, and the importance of reliable monitoring.

We express our gratitude to the Department of Computer Science and Engineering and the faculty members of the Operating Systems and Systems Programming course for providing the academic environment and resources required to complete this work.

We also thank our teammates, classmates, friends, and family members for their cooperation and encouragement. The project was completed through the combined contributions of Pravalika, Kommalapati Rishitha, and G.S. Hasini.

Pravalika **2520030402**
Kommalapati Rishitha **2520030202**
G.S. Hasini **2500030238**

Abstract

Linux Service Supervision is a project focused on observing the operational state of services running on a Linux system. Background services support important operating-system and application functions, and their availability is essential for reliable system operation. Manual inspection of several services can be repetitive, so a small automation utility can make routine checks easier.

This project uses Bash shell scripting together with systemd service-management utilities, especially systemctl, to query selected service units and display their reported states. The script organizes service checks, interprets the returned information, and presents a readable command-line summary. The work introduces service units, active and inactive states, failed services, command exit statuses, permissions, and basic troubleshooting through system logs.

The project provides practical experience in Linux administration and shell automation. It demonstrates how standard command-line tools can be combined to create a lightweight local supervision utility. The current scope is status monitoring; advanced functions such as remote monitoring, alert delivery, historical metrics, dashboards, and automatic recovery are treated as possible future enhancements unless separately implemented and validated.

Index

S. No.	Chapter / Topics / Subtopics	Page No.
	Acknowledgement	iv
	Abstract	v
3	1 Introduction	9–17
4	1.1 Background of Linux Service Supervision	
5	1.2 Need for Service Monitoring	
6	1.3 Problem Statement	
7	1.4 Project Aim and Objectives	
8	1.5 Scope and Importance	
9	1.6 Target Environment and Users	
10	2 OS Concepts and Background Study	18–37
11	2.1 Operating System and Linux Services	
12	2.2 systemd and Service Units	
13	2.3 Service States and Lifecycle	
14	2.4 systemctl, Logs and Permissions	
15	2.5 Proposed Approach	
16	3 System Requirements	38–46
17	3.1 Hardware Requirements	
18	3.2 Software Requirements	
19	3.3 Linux / Ubuntu Environment	
20	3.4 Bash and systemd Requirements	
21	4 Technology and Tools Used	47–57
22	4.1 Bash Shell Scripting	
23	4.2 systemctl and systemd	
24	4.3 Linux Commands and Utilities	
25	4.4 Shell Variables, Loops and Functions	
26	4.5 Error Handling	
27	5 System Architecture and Design	58–69

Index

S. No.	Chapter / Topics / Subtopics	Page No.
1	5.1 Overall System Architecture	
2	5.2 Service Monitoring Workflow	
3	5.3 Data Flow and Modules	
4	5.4 Input, Output and Error Design	
5	6 Implementation	70–89
6	6.1 Implementation Overview	
7	6.2 Project Directory Organization	
8	6.3 Algorithm and Pseudocode	
9	6.4 Service Status Query	
10	6.5 State Classification and Output	
11	6.6 Error Handling and Validation	
12	6.7 Implementation Limitations	
13	7 Execution and Demonstration	90–92
14	7.1 Execution Environment	
15	7.2 Running the Supervisor	
16	7.3 Demonstration of Service Status	
17	8 Results, Testing and Discussion	93–96
18	8.1 Results Summary	
19	8.2 Service Status Test	
20	8.3 Error and Boundary Testing	
21	8.4 Discussion and Limitations	
22	9 Conclusion, Future Scope and References	97–100
23	9.1 Conclusion and Objectives Achieved	
24	9.2 Learning Outcomes and Future Scope	
25	9.3 References	
26	9.4 Project Repository and Team Record	

CHAPTER 1 – INTRODUCTION

CHAPTER 1 – INTRODUCTION

Background of Linux Service Supervision

Linux systems run many background services that provide essential functions such as networking, logging, scheduling, remote access, and application hosting. A service may continue running without a user keeping a terminal open. Linux service supervision means observing these services, checking whether they are active or inactive, and identifying failures so that an administrator can respond.

Project relevance and key points

Linux Service Supervision focuses on monitoring services managed by systemd. The Bash-based supervisor uses systemctl information to inspect service states and present them in a readable form. The project connects operating-system concepts with a practical administration task.

- Services operate in the background and support system functions.
- systemd manages many services on modern Linux distributions.
- Regular status checks help identify service problems early.

CHAPTER 1 – INTRODUCTION

Need for Service Monitoring

An operating system may depend on several services at the same time. If an important service stops unexpectedly, users may experience unavailable applications, delayed jobs, or network problems. Checking every service manually is time-consuming, especially when a machine hosts multiple services.

Project relevance and key points

The project provides a simple, repeatable way to inspect service status from a shell script. It reduces the effort of repeatedly typing individual commands and makes the current state easier to review.

- Monitoring gives administrators visibility into service availability.
- A consistent check helps detect inactive or failed services.
- The report should distinguish observation from corrective action.

CHAPTER 1 – INTRODUCTION

Problem Statement

Linux administrators need a clear way to observe the state of system services. Manual inspection of individual units can be repetitive and may make it difficult to review several services together. A service that is inactive, failed, or repeatedly restarting may require attention.

Project relevance and key points

The project addresses this problem through a Linux service supervision script that queries systemd-managed services and reports their status. The implementation is intended as a lightweight command-line aid rather than a replacement for enterprise monitoring platforms.

- Identify the services that need supervision.
- Retrieve service state using systemctl.
- Display results in a readable and consistent format.

CHAPTER 1 – INTRODUCTION

Project Aim

The aim of the project is to develop a simple Linux service supervision utility using shell scripting and system administration commands. It checks the state of selected system services and helps a user understand whether each service is active, inactive, or in a failed condition.

Project relevance and key points

The main implementation focus is status monitoring through Bash and systemctl. The script acts as a small supervisory layer that gathers service information and displays it to the user.

- Use Bash to automate repeated service checks.
- Use systemctl as the interface to systemd service information.
- Present service status clearly for inspection.

CHAPTER 1 – INTRODUCTION

Objectives of the Project

The project objectives are to understand Linux service management, identify systemd-managed service units, automate status checks using a shell script, display service states, and interpret common service conditions. The work also aims to improve familiarity with Linux terminal commands and administrative workflows.

Project relevance and key points

These objectives are addressed through a command-line service supervision approach. Any action beyond status inspection, such as automatically restarting a service or sending alerts, is treated as a possible extension unless it is present in the team's actual script.

- Study Linux services and the systemd service manager.
- Automate service-state checks with Bash and systemctl.
- Report observations and identify services needing attention.

CHAPTER 1 – INTRODUCTION

Scope of the Project

The project scope is a command-line supervision utility for Linux services managed through systemd. It covers service discovery or selection, status retrieval, interpretation of common states, and display of results. The tool is intended for learning and small-scale system administration.

Project relevance and key points

The scope does not imply full enterprise monitoring, distributed monitoring across many servers, guaranteed automatic recovery, or a graphical dashboard. These may be considered as future improvements after the core script is validated.

- Focus on local Linux service supervision.
- Use standard system administration commands.
- Keep monitoring separate from privileged service-control operations.

CHAPTER 1 – INTRODUCTION

Importance of the Project

Service availability is an important part of reliable computing. Background services support logging, networking, scheduled tasks, remote access, and application operation. Understanding how to inspect these services helps students connect operating-system theory with real administration work.

Project relevance and key points

The project demonstrates how shell scripting can automate routine system checks. It also introduces service states, unit names, command exit codes, permissions, and log inspection.

- Builds practical Linux administration skills.
- Shows how automation reduces repetitive terminal work.
- Encourages early identification of service failures.

CHAPTER 1 – INTRODUCTION

Target Environment and Users

The intended environment is a Linux distribution that uses systemd, such as a current Ubuntu installation. The user runs the supervisor from a terminal with Bash and systemctl available. Some service details may require elevated privileges depending on the host configuration.

Project relevance and key points

The intended users are students, laboratory administrators, and Linux beginners who need a simple way to review service status. The tool is designed for local educational demonstrations.

- Linux with systemd is the expected platform.
- Bash provides the scripting environment.
- Users should understand basic terminal commands and permissions.

CHAPTER 1 – INTRODUCTION

Organization of the Report

This report introduces the service-supervision problem, explains Linux service and systemd concepts, lists system requirements, describes the tools, and presents the proposed architecture. It then explains the script workflow, execution procedure, testing observations, limitations, and future enhancements.

Project relevance and key points

The chapters follow the same sequence as the supplied KLH report template while replacing the original IPC subject matter with Linux Service Supervision. The final pages summarize learning outcomes and provide technical references and team details.

- Chapter 1 introduces the project and its objectives.
- Chapters 2–6 describe concepts, requirements, design, and implementation.
- Chapters 7–9 cover execution, testing, conclusions, and references.

CHAPTER 2 – OS CONCEPTS AND BACKGROUND STUDY

Operating System Overview

An operating system manages hardware resources and provides common services to applications. It schedules processes, manages memory, controls files, handles devices, and supports background services. Linux provides command-line tools and kernel interfaces that allow administrators to inspect and manage these components.

Project relevance and key points

The project studies the service-management layer of Linux. It uses systemd-related commands from a Bash script to observe services that run as part of the operating environment.

- The operating system coordinates processes and system resources.
- Background services provide long-running system functions.
- Service supervision supports system reliability.

CHAPTER 2 – OS CONCEPTS AND BACKGROUND STUDY

Program, Process, and Service

A program is a file containing executable instructions. A process is a running instance of a program. A service is commonly a long-running background process or a managed unit that provides a system or application function. Services may start during boot or on demand.

Project relevance and key points

The supervisor examines service units rather than treating every ordinary process as a service. This distinction is important because systemd tracks unit state and lifecycle information.

- A process is an executing program.
- A service is managed according to a service-management configuration.
- A service may create or manage one or more processes.

CHAPTER 2 – OS CONCEPTS AND BACKGROUND STUDY

Background Services in Linux

Many Linux tasks run without a graphical window or interactive terminal. Logging, time synchronization, network management, secure shell access, and scheduled jobs are examples of functions commonly supported by background services.

Project relevance and key points

The project makes these background components visible by checking their reported state. Service names should be selected carefully and verified on the target Linux machine.

- Background services operate independently of an interactive shell.
- Their state affects system functionality.
- Different Linux installations may have different service sets.

CHAPTER 2 – OS CONCEPTS AND BACKGROUND STUDY

Introduction to systemd

systemd is an init system and service manager used by many Linux distributions. It starts and supervises units, manages dependencies, and records state information. The systemctl command is the common command-line interface for interacting with systemd.

Project relevance and key points

The project uses systemctl to query service information. The exact output can vary by distribution and systemd version, so scripts should handle command failures and missing units.

- systemd manages services and other unit types.
- systemctl provides a command-line control interface.
- Unit configuration determines how a service is managed.

Service Units

A unit is a resource or task managed by systemd. Common unit types include service, socket, timer, mount, target, and path units. A service unit generally describes how a background service is started, stopped, and supervised.

Project relevance and key points

The project concentrates on service units. Filtering by unit type avoids mixing service records with timers, mounts, and other systemd resources.

- Unit files describe managed resources.
- Service units commonly use the .service suffix.
- systemctl can list and inspect units.

Service States

A service can be reported as active, inactive, failed, activating, deactivating, or in another transitional state. The active state indicates that systemd considers the unit active; it does not always prove that the application is functioning correctly at every level.

Project relevance and key points

The supervisor should display the state returned by systemctl without making unsupported assumptions. A failed state is an indication that further inspection may be needed.

- active and inactive describe different service conditions.
- failed indicates an unsuccessful or failed unit state.
- State interpretation should consider the service's purpose.

Active and Inactive Services

An active service is considered running or otherwise active according to its unit type. An inactive service is not currently active. Some services are intentionally inactive until triggered, so inactive does not automatically mean that the system is broken.

Project relevance and key points

The project reports status so a user can decide whether the observed state is expected. A monitoring script should avoid marking every inactive unit as a critical failure.

- Service expectations differ by machine and workload.
- A service can be enabled but currently inactive.
- Status should be interpreted in context.

Failed Services

A service may enter a failed state when its process exits unsuccessfully, startup fails, or a configured condition is not met. The reason may be visible in systemd status output or the journal.

Project relevance and key points

The supervisor helps identify failed units. Diagnosis still requires examining the service logs, configuration, dependencies, and recent system changes.

- Failure state is a useful warning signal.
- journalctl can provide supporting diagnostic information.
- The script should show failures clearly.

Service Lifecycle

A typical service lifecycle includes loading its unit configuration, starting, becoming active, running, stopping, and possibly entering a failed state. systemd coordinates these transitions and records the resulting state.

Project relevance and key points

The project's monitoring flow observes the current lifecycle state. It does not need to control every transition to provide useful status information.

- Services can transition between several states.
- systemd coordinates lifecycle operations.
- A status snapshot represents a point in time.

systemctl Command

systemctl is used to inspect and control the systemd manager. Commands such as systemctl status, systemctl is-active, systemctl list-units, and systemctl list-unit-files provide different views of service information.

Project relevance and key points

The supervisor uses systemctl as its primary service-information source. Choosing the right command depends on whether the script needs detailed status, a concise state, or a list of units.

- status provides detailed information about one unit.
- is-active provides a concise active-state result.
- list-units can display loaded units.

Service Discovery

Service discovery is the process of identifying which service units exist on a Linux machine. A system may contain distribution services, hardware-related units, application services, and user-created units.

Project relevance and key points

The project may inspect a defined list or retrieve a list from systemctl. A fixed list is easier to demonstrate, while dynamic discovery is more flexible but requires filtering.

- Service availability varies by Linux installation.
- Use systemctl list-units or list-unit-files to inspect units.
- Filter results to service units when required.

Enabled and Disabled Services

Enabled describes whether a unit is configured to start automatically under specified conditions. Disabled means it is not enabled in that way. Enabled status is different from whether a service is currently running.

Project relevance and key points

The supervisor should not confuse startup configuration with current runtime state. A service can be enabled but inactive, or disabled but active because it was started manually or by another unit.

- enabled is not the same as active.
- disabled does not necessarily mean failed.
- Use the appropriate systemctl query for each property.

Service Dependencies

A service may depend on other units or be ordered relative to them. systemd uses dependency and ordering relationships to coordinate startup and shutdown. A service may fail if a required dependency is unavailable.

Project relevance and key points

When a monitored service fails, its dependencies may be relevant to diagnosis. The project can identify the affected unit, while detailed dependency analysis remains a diagnostic step.

- Dependencies influence service startup.
- Ordering controls when units are started or stopped.
- Dependency failures can affect dependent services.

Logs and the Journal

The systemd journal stores system and service messages. journalctl can display logs for the entire system or filter messages for a particular unit. Logs help explain why a service did not start or why it stopped.

Project relevance and key points

The supervision script identifies service states; journalctl is a supporting command for investigating unusual states. Log access and retention depend on system configuration.

- Logs provide context beyond a status label.
- journalctl -u can filter by unit.
- A failed service should be investigated using relevant logs.

Permissions and Privileges

Linux uses user and group permissions to protect system resources. Querying service status is often available to ordinary users, while changing system services usually requires administrative privileges. The exact permissions depend on policy and distribution configuration.

Project relevance and key points

The project should run monitoring commands with the minimum permissions needed. Privileged operations should not be added casually to a monitoring script.

- Use least privilege for routine monitoring.
- Administrative actions may require sudo.
- Do not embed passwords in scripts.

Shell Scripting Fundamentals

A shell script is a text file containing commands interpreted by a shell. Bash supports variables, loops, conditions, functions, command substitution, and exit-status handling. These features allow repeated system checks to be automated.

Project relevance and key points

The project uses Bash to organize service checks and format the output. Clear functions and careful quoting make the script easier to maintain.

- #!/bin/bash selects the Bash interpreter.
- Variables and loops support repeated checks.
- Exit codes should be checked.

Command Exit Status

Linux commands return an exit status when they finish. A zero status usually indicates success, while a non-zero value indicates an error or a condition that should be interpreted by the caller. systemctl commands may use non-zero statuses for inactive or unavailable units.

Project relevance and key points

The supervisor should interpret command results carefully. A non-zero result should not automatically be treated as a script crash; it may represent a meaningful service state.

- Use $? or conditional command execution to check results.
- Distinguish inactive from command execution failure.
- Display useful error messages.

Existing Manual Approach and Limitations

Without automation, an administrator may run systemctl status separately for each service. This approach is understandable for a few units but becomes repetitive when many services need to be reviewed. Manual checks also make it harder to apply a consistent output format.

Project relevance and key points

The proposed script automates repeated status queries and displays the results in one place. It remains a lightweight local utility and does not replace centralized monitoring.

- Manual inspection is suitable for occasional checks.
- Repeated commands consume time.
- Automation improves consistency.

Proposed Service Supervision Approach

The proposed approach uses a Bash script to query selected systemd services, obtain their current states, and display a summary. The user launches the script from a Linux terminal and reviews the reported service conditions.

Project relevance and key points

The design emphasizes read-only monitoring. Any start, stop, restart, alerting, or remote-host capability should be identified as future scope unless separately implemented and tested.

- Collect service names or discover service units.
- Query state using systemctl.
- Display a clear status summary.

Advantages of the Proposed Approach

A shell-based supervisor is lightweight, uses tools already available on many Linux systems, and is easy to inspect. It reduces repetitive command entry and provides a repeatable method for checking service states.

Project relevance and key points

The project demonstrates practical automation without requiring a database or external monitoring server. Its simplicity makes it suitable for operating-systems coursework.

- Uses standard Linux utilities.
- Can be modified for a different service list.
- Provides a foundation for more advanced monitoring.

CHAPTER 3 – SYSTEM REQUIREMENTS

Hardware Requirements

The project has modest hardware requirements because it runs shell commands and reads service-manager information. A desktop or laptop capable of running a supported Linux distribution is sufficient. No special sensor, controller, or graphics hardware is required.

Project relevance and key points

The supervisor operates on the local Linux host. System performance depends mainly on the operating system and the number of services queried.

- A standard computer is sufficient.
- No dedicated hardware is required.
- A stable Linux environment is recommended.

CHAPTER 3 – SYSTEM REQUIREMENTS

Processor and Memory

Any processor supported by the selected Linux distribution can run the Bash script. The script itself uses little memory because it processes command output and status values rather than large datasets.

Project relevance and key points

The monitoring workload is small for a short service list. Querying very large numbers of units repeatedly can increase command overhead, so the interval and scope should be reasonable.

- CPU requirements are low.
- Memory use is generally modest.
- Avoid unnecessarily frequent polling.

CHAPTER 3 – SYSTEM REQUIREMENTS

Storage Requirements

Storage is required for the Bash source file, documentation, and any optional output logs. The script does not need a large database for basic status inspection. If logs are saved, the project should consider file permissions and log growth.

Project relevance and key points

The current project is a lightweight command-line utility. Persistent history or long-term metrics would require an additional storage design and are treated as future enhancement unless implemented.

- Store the script in a suitable project directory.
- Keep optional logs separate from source code.
- Manage log size if persistent logging is added.

CHAPTER 3 – SYSTEM REQUIREMENTS

Software Requirements

The software environment requires Linux with systemd, Bash, systemctl, and a terminal. Standard utilities such as grep, awk, sed, and date may be used for formatting or filtering if included in the script.

Project relevance and key points

The project relies on system tools rather than a separate application server. Command availability should be verified on the machine used for demonstration.

- Linux distribution using systemd.
- Bash shell and systemctl command.
- Optional text-processing utilities.

CHAPTER 3 – SYSTEM REQUIREMENTS

Linux and Ubuntu Environment

Linux provides the process, filesystem, permission, and service-management facilities used by the project. Ubuntu commonly uses systemd as its init and service manager, making systemctl available for service inspection.

Project relevance and key points

The report uses Ubuntu/Linux as the intended environment. Other systemd-based distributions may also work, but command output and available service names can differ.

- Verify the host uses systemd.
- Use a terminal in the project directory.
- Service names may differ across distributions.

CHAPTER 3 – SYSTEM REQUIREMENTS

Bash Installation and Verification

Bash is installed by default on many Linux distributions. Its version can be checked from a terminal. A script should use a valid shebang and executable permissions before it is run directly.

Project relevance and key points

The project uses Bash syntax for automation. If the script is executed with bash script.sh, executable permission is not essential; direct execution requires the executable bit.

- Check Bash availability with bash --version.
- Use a correct shebang line.
- Set execute permission when needed.

systemd and systemctl Availability

The systemctl command communicates with the systemd manager. A host that does not use systemd may not support the same commands or behavior. Containers and some WSL configurations may also have limitations depending on how they are configured.

Project relevance and key points

Before running the project, confirm that systemctl can communicate with the service manager. This prevents confusion between script errors and an unsupported environment.

- Run systemctl --version to check availability.
- Check whether systemd is PID 1 where relevant.
- Use a systemd-enabled environment for the demonstration.

CHAPTER 3 – SYSTEM REQUIREMENTS

Terminal and Shell

The terminal provides the command-line interface for launching the supervisor and viewing its output. The shell interprets commands, expands variables, runs loops, and reports command exit statuses.

Project relevance and key points

The terminal is the primary user interface for this project. It allows the evaluator to observe service names, state labels, and any error messages.

- Open a terminal in the project directory.
- Run the script using Bash or direct execution.
- Keep output readable and well aligned.

Permissions and Working Directory

The project files should be stored in a directory where the user can read and execute the script. Monitoring service state often does not require root access, but access to some details may depend on host policy. Service-control commands generally require elevated privileges.

Project relevance and key points

The supervisor should request only the permissions necessary for its monitoring purpose. Avoid placing project files in protected system directories.

- Use a user-owned project folder.
- Check file permissions with ls -l.
- Use sudo only when genuinely required.

CHAPTER 4 – TECHNOLOGY AND TOOLS USED

Bash Shell Scripting

Bash is a command interpreter that supports variables, conditions, loops, functions, and command substitution. It is widely used for Linux administration and automation. A script can call systemctl and process its output.

Project relevance and key points

Bash is the main implementation technology for the service supervisor. It provides a compact way to repeat status checks and format the results.

- Use functions to separate script tasks.
- Quote variables to avoid word-splitting problems.
- Keep messages clear and actionable.

systemctl Utility

systemctl is the command-line tool used to query and control systemd. It can show the status of one unit, report whether a service is active, list loaded units, and inspect unit-file enablement.

Project relevance and key points

The supervisor uses systemctl to retrieve service information. The script should use concise output modes where practical and handle units that are missing or unavailable.

- systemctl status UNIT shows detailed status.
- systemctl is-active UNIT returns a concise state.
- systemctl list-units --type=service lists service units.

CHAPTER 4 – TECHNOLOGY AND TOOLS USED

systemd Service Manager

systemd manages service units and other resources during system startup and runtime. It coordinates dependencies, unit activation, and state tracking. It also provides integration with the system journal.

Project relevance and key points

Understanding systemd helps explain where the status information comes from. The script does not replace systemd; it reads and summarizes information provided by it.

- systemd supervises units.
- systemctl communicates with the manager.
- Service state is maintained by the service manager.

CHAPTER 4 – TECHNOLOGY AND TOOLS USED

Linux Commands and Utilities

Commands such as pwd, cd, ls, chmod, bash, systemctl, and journalctl are useful for running and troubleshooting the project. Text-processing utilities may be used to filter command output.

Project relevance and key points

The command-line tools provide the execution environment and supporting diagnostics. Only commands actually used by the script should be described as implementation dependencies.

- pwd and ls help locate project files.
- chmod can make a script executable.
- journalctl helps inspect service logs.

Shell Variables and Arrays

Variables store values such as service names, command results, and display labels. Bash arrays can store a list of services so that a loop can check each service in sequence.

Project relevance and key points

A service list can be maintained in an array or read from a configuration file. The implementation should quote expansions and handle empty values safely.

- Variables improve readability.
- Arrays support checking multiple services.
- Validate service names before querying them.

Conditional Statements

Bash if, elif, and else statements allow a script to select different output based on command results. A monitoring script can distinguish active, inactive, failed, and unknown states.

Project relevance and key points

Conditional logic is central to interpreting systemctl results. The script should report the returned state rather than assuming every non-zero exit code means the same thing.

- Use if statements for state decisions.
- Provide a fallback for unknown results.
- Keep state labels consistent.

Loops and Repeated Checks

A loop repeats a command for each item in a list. A for loop is suitable for checking a known list of service names. A while loop can be used for continuous monitoring when an interval is defined.

Project relevance and key points

The project uses repetition to inspect multiple services without manually typing the same command for each one. Continuous polling should be an optional behavior and should have a controlled interval if implemented.

- Loops reduce repeated code.
- Use a defined list or filtered service list.
- Avoid tight loops that consume unnecessary resources.

Functions in Bash

Functions group related commands into reusable units. A script may define functions for checking one service, printing a heading, or displaying a summary. This structure improves readability and makes testing easier.

Project relevance and key points

A modular script is easier to maintain than a long sequence of repeated commands. Function names should describe their purpose clearly.

- Use functions for repeated tasks.
- Pass service names as arguments.
- Keep output and checking logic organized.

Command Substitution and Pipes

Command substitution captures the output of a command into a variable. Shell pipelines send the output of one command to another for filtering or formatting. These features are useful for extracting a concise service state.

Project relevance and key points

The supervisor may use command substitution to store systemctl output before displaying it. Pipelines should be designed carefully so that command failures are not hidden.

- $(command) captures command output.
- Pipes connect command output to another utility.
- Check pipeline exit behavior where needed.

Exit Codes and Error Handling

A shell command returns an exit status. Bash conditions can test this status directly. A script should handle missing commands, unavailable service units, and permission-related errors without terminating unexpectedly.

Project relevance and key points

Error handling helps the supervisor distinguish a valid service state from a failed query. Clear messages are more useful than silently ignoring errors.

- Check command results.
- Handle missing or invalid unit names.
- Avoid displaying misleading status labels.

Text Formatting and Output

A monitoring tool should present results in a form that is easy to scan. Consistent columns for service name and status can help users compare several units. Terminal colours may improve visibility if used with accessible text labels.

Project relevance and key points

The project emphasizes readable command-line output. Colour should supplement, not replace, the written state so that output remains understandable in terminals without colour support.

- Align service names and states.
- Use clear labels such as ACTIVE or FAILED.
- Keep output useful when redirected to a file.

CHAPTER 5 – SYSTEM ARCHITECTURE AND DESIGN

Overall System Architecture

The project follows a simple command-line architecture. The user starts a Bash script, the script prepares a list of services, systemctl queries the systemd manager, and the script displays the returned state information. The Linux service manager remains responsible for the actual service lifecycle.

Project relevance and key points

The architecture separates the user interface, monitoring logic, command execution, and output formatting. The script observes services rather than replacing the systemd manager.

- User launches the supervisor script.
- Bash invokes systemctl for each selected unit.
- Results are interpreted and displayed in the terminal.

CHAPTER 5 – SYSTEM ARCHITECTURE AND DESIGN

User Interaction Design

The user interacts with the project through a terminal. The script may display a heading, explain what is being checked, and print the service results. If a menu is included in the team's version, its options should be documented exactly as implemented.

Project relevance and key points

A simple terminal interface keeps the project lightweight. Input should be validated so that an unexpected service name does not produce confusing output.

- Provide a clear start message.
- Validate user-supplied service names if accepted.
- Display completion or error messages.

Service List Design

The supervisor needs to know which services to inspect. The list may be fixed in the script for a controlled demonstration or obtained dynamically from systemctl. A fixed list gives predictable output; dynamic discovery covers more units but needs filtering.

Project relevance and key points

The report describes service selection as a configurable design choice. The actual demonstration should use the same service names that are present on the team's Linux environment.

- Choose services that exist on the target machine.
- Keep the list easy to update.
- Handle missing units gracefully.

Monitoring Data Flow

The data flow begins when the user launches the script. The script identifies a service, calls systemctl, captures the state or status output, interprets the result, and prints a line for the user. The process repeats until all selected services are checked.

Project relevance and key points

This sequence makes it clear that the script is a monitoring layer. It reads status information from systemd and does not directly inspect or modify kernel service state.

- Input: service name or service list.
- Processing: systemctl query and state interpretation.
- Output: service name with observed state.

Service Status Module

The status module queries a service and obtains its current state. A concise query such as systemctl is-active can be used when only the active state is required. A detailed status query can provide additional diagnostic context.

Project relevance and key points

This module is the core of the project. It should return a meaningful label for active, inactive, failed, or unknown results.

- Query one service at a time or in a controlled batch.
- Capture the result safely.
- Do not treat every inactive service as a failure.

Service Discovery Module

A discovery module can obtain a list of loaded service units from systemctl. It may filter the list to avoid showing unrelated unit types. Dynamic discovery is useful when the available services are not known in advance.

Project relevance and key points

If the current script uses a predefined service list, dynamic discovery can be documented as a possible extension rather than a completed feature.

- Use service-specific listing options.
- Filter output carefully.
- Avoid assuming all systems expose identical units.

Status Classification Module

Status classification maps command output into readable categories. For example, active may be displayed as RUNNING, inactive as STOPPED, failed as FAILED, and an unrecognized result as UNKNOWN. The exact labels should remain consistent throughout the application.

Project relevance and key points

The classification module makes raw command output easier to understand. It should preserve the difference between an inactive unit and an unsuccessful query.

- Define state labels once.
- Provide a fallback category.
- Include raw details when a state is ambiguous.

Output and Summary Module

The output module formats each service result and may provide a count of active, inactive, or failed services. A summary can help the user identify the overall condition without reading every line in detail.

Project relevance and key points

A summary is useful only when its counts are based on successful queries. Unknown or unavailable results should be counted separately instead of being forced into another category.

- Show service names and states.
- Use consistent formatting.
- Summarize counts only when results are valid.

Error Handling Design

Errors may occur when systemctl is unavailable, a service name is incorrect, the service manager cannot be contacted, or permissions restrict access. The script should report these conditions clearly and continue checking other services where safe.

Project relevance and key points

Error handling improves the reliability of the demonstration. It also prevents a single invalid unit from hiding the status of the remaining services.

- Validate command availability at startup.
- Report query failures separately from service states.
- Continue safely after an individual service error.

Security and Permission Design

A monitoring script should use the minimum privileges required to read service state. It should not store administrator passwords or execute arbitrary user input as a privileged command. Any service-control capability needs deliberate permission checks.

Project relevance and key points

The project is designed around status observation. Keeping monitoring read-only reduces risk and makes the tool suitable for a classroom demonstration.

- Avoid hard-coded credentials.
- Validate any supplied service name.
- Use privileged commands only when necessary.

CHAPTER 5 – SYSTEM ARCHITECTURE AND DESIGN

Architecture Limitations

A local Bash script normally supervises services on the machine where it runs. It may not provide historical metrics, remote monitoring, alert delivery, or a graphical dashboard. Its output is generally a snapshot rather than a continuous health history.

Project relevance and key points

These boundaries define the current project scope. Advanced monitoring can be added later without overstating the current implementation.

- Local status checks are not distributed monitoring.
- A service state is not a complete application health test.
- Historical trend analysis requires additional storage.

CHAPTER 5 – SYSTEM ARCHITECTURE AND DESIGN

Input and Output Design

Inputs may consist of a configured list of service unit names or a user-supplied name. Outputs consist of the observed state and any relevant error message. If the script accepts interactive input, it should reject blank or malformed values.

Project relevance and key points

A consistent input/output design makes the project easier to demonstrate and test. The report should match the final script's actual input method.

- Keep service identifiers in their systemd unit form.
- Display output in a consistent layout.
- Show clear feedback for invalid input.

CHAPTER 6 – IMPLEMENTATION

Implementation Overview

The implementation is based on a Bash script that queries systemd-managed services using systemctl. The script organizes the selected service names, retrieves status information, interprets the result, and prints a summary in the terminal. This provides a practical example of Linux service supervision.

Project relevance and key points

The implementation should be demonstrated on a Linux system with systemd. The exact service list and output should be taken from the team's tested script and screenshots.

- Bash provides the automation logic.
- systemctl supplies service-state information.
- The terminal displays the supervision results.

CHAPTER 6 – IMPLEMENTATION

Project Directory Organization

A simple project directory can contain the main shell script, optional configuration file, README documentation, and sample output or screenshots. Keeping source files separate from generated logs makes the project easier to maintain.

Project relevance and key points

The team should use the actual filenames from its repository. The report's suggested organization is a logical structure and should not be mistaken for files that do not exist.

- Keep the main script in the project root or scripts folder.
- Document how to run the script.
- Store optional output separately.

CHAPTER 6 – IMPLEMENTATION

Algorithm for Service Supervision

Step 1: start the Bash script. Step 2: verify that systemctl is available. Step 3: prepare or discover the service list. Step 4: query each service. Step 5: classify the returned state. Step 6: display the result. Step 7: finish after all selected services are checked.

Project relevance and key points

This algorithm describes the monitoring workflow. It is suitable for a one-time status report and can be extended with a timed loop if continuous monitoring is implemented.

- Check prerequisites before querying services.
- Process each service independently.
- Report state and query errors clearly.

CHAPTER 6 – IMPLEMENTATION

Pseudocode for the Supervisor

START; verify Bash and systemctl; if systemctl is unavailable, print an error and stop; load service list; for each service, query its active state; if active, print ACTIVE; otherwise inspect or classify the returned state; print the result; continue until all services are checked; END.

Project relevance and key points

Pseudocode provides a language-independent view of the script. The final Bash implementation should follow the same logical order and handle unexpected command results.

- Initialize the monitoring environment.
- Loop through service names.
- Display the result for every attempted check.

Checking systemctl Availability

Before monitoring begins, the script can verify that systemctl exists using command -v systemctl. It may also check whether the system manager is reachable. This helps distinguish a missing command from an unavailable systemd environment.

Project relevance and key points

A prerequisite check prevents confusing output when the script is run in an unsupported environment, such as a container without a running systemd manager.

- Use command -v to locate systemctl.
- Report unsupported environments clearly.
- Do not continue as if queries succeeded.

Querying a Service State

A concise way to query whether a service is active is systemctl is-active SERVICE.service. The returned text can be captured in a variable. The command's exit status should also be considered because inactive or failed states may return a non-zero status.

Project relevance and key points

The supervisor uses this query pattern to obtain a state for each service. The unit name should be valid and should correspond to a service installed on the host.

- Use the correct unit name.
- Capture output and exit status separately when needed.
- Handle inactive, failed, and unknown results.

Using systemctl status

systemctl status SERVICE.service provides a human-readable status view that can include the loaded state, active state, main process, and recent journal lines. It is useful for manual inspection and troubleshooting.

Project relevance and key points

The project may use this command as a detailed view or as a validation step during testing. For machine-readable logic, concise commands are generally easier to parse than the full status display.

- Use status for detailed manual inspection.
- Avoid parsing decorative output unnecessarily.
- Check the service's actual unit name.

Looping Through Multiple Services

A Bash for loop can run the same status query for each service in an array. The loop avoids duplicated code and makes it easy to add or remove service names. Quoting each variable expansion protects names containing unexpected characters.

Project relevance and key points

This approach supports supervision of multiple services in a single execution. The number of services should remain manageable for a quick classroom demonstration.

- Store service names in an array or configuration file.
- Use quoted variable expansions.
- Continue through the list after an individual query error.

Interpreting Service States

The script should map the state returned by systemctl into understandable output. Active, inactive, failed, activating, deactivating, and unknown results should not be confused. The project may use simplified labels for presentation while retaining the underlying meaning.

Project relevance and key points

Correct interpretation is essential to avoid false alarms. Inactive services can be normal when they are on-demand or not required at the moment.

- Display state names consistently.
- Separate failed state from inactive state.
- Treat unrecognized output as unknown.

Displaying a Status Table

A terminal table can contain columns for service name and status. printf is useful for consistent spacing. If service names have different lengths, fixed-width formatting or truncation can help maintain alignment.

Project relevance and key points

Readable output is a central part of the user experience. State text should remain visible even when terminal colour is unavailable.

- Use headings for columns.
- Align service names and state labels.
- Avoid relying only on colours.

Counting Service States

A summary may count how many checked services are active, inactive, failed, or unknown. Counters can be initialized before the loop and incremented after each valid query.

Project relevance and key points

Counts provide a quick overview of the checked list. They should be calculated from the actual results and should not include failed queries as ordinary inactive services.

- Initialize counters before iteration.
- Increment the matching counter only after classification.
- Print totals after all checks finish.

Handling Invalid Service Names

A service name may be misspelled or may not exist on the machine. systemctl can report that the unit could not be found. The script should show a clear message and proceed to the next service when appropriate.

Project relevance and key points

This protects the monitoring run from one invalid entry. A configurable service list should be checked against the host before a demonstration.

- Validate service names where possible.
- Report missing units as unavailable or unknown.
- Do not silently skip invalid entries.

Logging and Diagnostic Output

A script can print status to the terminal and optionally write a timestamped record to a log file. Logging supports later review, but it introduces file permissions, retention, and disk-space considerations.

Project relevance and key points

If persistent logging is not part of the current implementation, it should be described as an extension. Terminal output is sufficient for a basic status-check demonstration.

- Include timestamps if records are saved.
- Protect log files with suitable permissions.
- Plan log rotation for long-running use.

CHAPTER 6 – IMPLEMENTATION

Continuous Monitoring

Continuous monitoring repeats service checks at a chosen interval. A loop with sleep can implement simple periodic polling. The interval should balance responsiveness with system overhead.

Project relevance and key points

The current project scope should be described accurately: if the script performs a single scan, continuous polling is future work. If it loops continuously, the interval and exit method should be documented.

- Choose a sensible polling interval.
- Provide a way to stop the monitoring loop.
- Avoid excessive command execution.

Optional Service Recovery

A supervisor can be extended to restart a service after a failure, but automatic recovery changes system state and may affect dependent applications. A restart should only be attempted under a clearly defined policy and with suitable privileges.

Project relevance and key points

Automatic recovery is not assumed to be part of the basic monitoring implementation. It is a future enhancement that requires safeguards, audit output, and testing.

- Separate detection from remediation.
- Require explicit policy for restart actions.
- Record any corrective action and its result.

CHAPTER 6 – IMPLEMENTATION

Source Code Structure

A maintainable Bash script usually begins with a shebang, sets safe shell options where appropriate, defines variables or arrays, declares helper functions, checks prerequisites, and then performs the monitoring loop. Comments explain non-obvious decisions.

Project relevance and key points

The report describes the expected structure of a readable supervisor script. The team should align the final code listing with its actual source file.

- Use a clear shebang.
- Organize repeated logic into functions.
- Comment important checks and assumptions.

CHAPTER 6 – IMPLEMENTATION

Compilation and Execution

Bash scripts do not require compilation into a native executable. They can be run with bash script.sh or made executable and launched directly. Syntax can be checked using bash -n script.sh before execution.

Project relevance and key points

The project uses shell execution rather than GCC compilation. This distinction is important when demonstrating the tool to an evaluator.

- Run bash -n to check syntax.
- Use chmod +x when direct execution is desired.
- Run the script from its project directory.

CHAPTER 6 – IMPLEMENTATION

Implementation Limitations

The basic supervisor provides a local view of service state. It does not automatically prove application-level health, collect long-term performance metrics, or monitor remote hosts. Status output can also vary across Linux versions and configurations.

Project relevance and key points

These limitations are normal for a small academic Bash utility. They define what the project demonstrates and what would require additional components.

- A service may be active while its application is unhealthy.
- The tool depends on systemd and systemctl.
- Historical and remote monitoring are outside the basic scope.

Code Safety and Validation

Shell scripts should quote variables, validate inputs, check command availability, and avoid evaluating arbitrary text as shell code. Commands should be invoked with clear arguments and errors should be reported without exposing sensitive data.

Project relevance and key points

These practices improve the reliability and safety of the supervisor. Monitoring should remain read-only unless a separately reviewed service-control feature is required.

- Quote variable expansions.
- Avoid eval for user-provided values.
- Check external command results.

CHAPTER 6 – IMPLEMENTATION

Implementation Summary

The implementation connects Bash scripting with systemd service information. It automates the repeated process of querying service state and presenting the result. The design is small enough to understand while demonstrating a real Linux administration task.

Project relevance and key points

The implementation chapter documents the core supervision workflow and its operational boundaries. Actual screenshots and source-code excerpts should be inserted from the team's final tested version.

- Automates status queries.
- Presents results in a readable format.
- Provides a base for future alerting and recovery features.

CHAPTER 7 – EXECUTION AND DEMONSTRATION

Execution Environment

The project should be executed on a Linux system using systemd and Bash. Before running the script, verify the operating system, shell, and systemctl command. Confirm that the project files are present in the current working directory.

Project relevance and key points

A native Ubuntu or other systemd-based Linux environment is recommended. Some WSL or container configurations may not run systemd services in the same way as a full Linux installation.

- Check the environment before execution.
- Open a terminal in the project directory.
- Verify systemctl can communicate with systemd.

CHAPTER 7 – EXECUTION AND DEMONSTRATION

Running the Supervisor Script

The script can be launched with bash supervisor.sh, replacing the filename with the actual project script name. If executable permission is set, it can also be run as ./supervisor.sh. The terminal displays the service checks and their results.

Project relevance and key points

For the demonstration, use the exact script filename and service list in the team's repository. Keep the terminal visible so the evaluator can observe the status output.

- Run the script from the correct directory.
- Use the actual project filename.
- Capture the complete terminal output for documentation.

CHAPTER 7 – EXECUTION AND DEMONSTRATION

Demonstration of Service Status

A demonstration should show the script checking a small set of known services and displaying their current states. The evaluator can compare one or two results with systemctl status SERVICE to verify that the script reports the system manager's information correctly.

Project relevance and key points

The output is a snapshot of the machine at the time of execution. Service states may differ between computers, so the report should not claim fixed active or failed states without a matching screenshot.

- Select services available on the test machine.
- Compare output with systemctl status.
- Record the date and environment for reproducibility.

CHAPTER 8 – RESULTS, TESTING AND DISCUSSION

Results Summary

The project demonstrates a Bash-based approach to Linux service supervision. The script queries service information through systemctl and presents the observed state to the user. This shows how standard Linux administration commands can be combined into a repeatable monitoring workflow.

Project relevance and key points

Results should be supported by screenshots or terminal records from the team's actual execution. The service states shown depend on the test machine and can change over time.

- The script automates service-state inspection.
- systemctl provides the status information.
- The terminal output is the primary demonstration evidence.

Service Status Test and Interpretation

A normal test selects a service that exists on the host and runs the supervisor. The returned state should agree with a direct systemctl query performed at approximately the same time. Differences may occur if the service changes state between checks.

Project relevance and key points

The comparison validates that the script is reading service-manager information correctly. A test record should identify the service name, supervisor output, direct query output, and observation.

- Expected result: reported state matches systemctl.
- Repeat the test if the service changes during execution.
- Record the actual result rather than assuming it.

Error and Boundary Testing

Boundary tests should include an invalid service name, an unavailable systemctl command, an environment without a running systemd manager, and a service whose state is inactive or failed. The script should report these conditions clearly without producing misleading results.

Project relevance and key points

Testing should distinguish an expected non-active state from a genuine query failure. The script should continue safely where possible and return an appropriate exit status if the overall run cannot proceed.

- Test a valid service name.
- Test a missing or misspelled unit.
- Test behavior when systemctl cannot be reached.

CHAPTER 8 – RESULTS, TESTING AND DISCUSSION

Discussion and Limitations

The project provides a practical introduction to service supervision using shell scripting. It improves consistency compared with repeated manual commands, but it is not a full monitoring platform. A service's systemd state is only one indicator and may not reflect end-to-end application health.

Project relevance and key points

The results should be interpreted within the project's academic scope. Continuous monitoring, alert delivery, remote hosts, historical metrics, and automatic recovery require additional design and validation.

- Status monitoring is useful but not complete health monitoring.
- Results depend on the Linux host configuration.
- Future features should be tested separately.

CHAPTER 9 – CONCLUSION, FUTURE SCOPE AND REFERENCES

Conclusion and Objectives Achieved

The project, Linux Service Supervision, demonstrates how Bash scripting and systemctl can be used to inspect the state of Linux services. It connects operating-system concepts with a practical administrative task: checking whether selected background services are active, inactive, failed, or otherwise unavailable.

Project relevance and key points

The central objective is to automate service-state inspection and present the results clearly. The project provides practical experience with shell scripting, systemd, command exit statuses, permissions, and Linux troubleshooting.

- Introduced Linux services and systemd.
- Used Bash to automate status checks.
- Demonstrated service-state reporting.

CHAPTER 9 – CONCLUSION, FUTURE SCOPE AND REFERENCES

Learning Outcomes and Future Scope

The project provides experience in Linux terminal usage, Bash control structures, systemctl queries, service-state interpretation, and basic error handling. It also highlights the importance of testing on the actual target distribution because service names and states vary.

Project relevance and key points

Future development may add periodic monitoring, timestamped logs, configurable service lists, email or desktop alerts, a dashboard, remote-host checks, and carefully controlled recovery actions. These are extensions and should not be described as completed unless implemented.

- Add continuous monitoring with a configurable interval.
- Introduce alerts and historical logs.
- Consider a dashboard and remote monitoring.

CHAPTER 9 – CONCLUSION, FUTURE SCOPE AND REFERENCES

References and Technical Documentation

The project relies on Linux and systemd documentation for service management, Bash documentation for shell syntax, and manual pages for commands used during implementation. Official documentation should be consulted to confirm command options and behavior for the installed version.

Project relevance and key points

References should match the tools actually used in the final script. The following sources provide the technical foundation for Bash scripting and systemd service inspection.

- systemd documentation and systemctl(1) manual page.
- Bash Reference Manual and bash(1) manual page.
- Ubuntu/Linux documentation and journalctl(1) manual page.

CHAPTER 9 – CONCLUSION, FUTURE SCOPE AND REFERENCES

Project Repository and Team Record

The project report is titled Linux Service Supervision. It documents a Bash-based utility for monitoring Linux service states through systemctl. The team details below are reproduced from the information supplied for this report.

Project relevance and key points

The source-code repository URL was not provided with the project details in this request. Add the verified repository link here before final academic submission if the team maintains one.

- Project Title: Linux Service Supervision.
- Project Guide: Harika.
- Team Members: Pravalika – 2520030402; Kommalapati Rishitha – 2520030202; G.S. Hasini – 2500030238.